



UNIVERSITAS
AMIKOM
YOGYAKARTA

2026

Modul Praktikum Pengenalan Perancangan Web



Disclaimer

Hanya dipergunakan di Lingkungan Internal
Universitas AMIKOM Yogyakarta

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas berkah dan karunia-Nya, sehingga penyusunan Modul Praktikum Pengenalan Perancangan Web (PPW) ini dapat diselesaikan. Modul ini dikembangkan sebagai pedoman dalam melakukan kegiatan praktikum mata kuliah Praktikum Pengenalan Perancangan Web yang merupakan salah satu mata kuliah wajib pada Jurusan S1 Sistem Informasi Universitas AMIKOM Yogyakarta.

Modul ini diharapkan dapat membantu mahasiswa/i, pengembang, atau pun rekan pengajar dalam mempersiapkan dan melaksanakan kegiatan praktikum perkuliahan dengan lebih baik serta menambah wawasan tentang pengenalan web. Pada setiap topik bahasan modul ini telah ditetapkan tujuan materi yang disampaikan dan semua tugas yang harus dilakukan oleh mahasiswa/i serta teori singkat untuk memperdalam pemahaman mahasiswa/i mengenai materi yang dibahas.

Penyusun meyakini bahwa dalam pembuatan modul ini masih jauh dari sempurna. Oleh karena itu penyusun mengharapkan kritik dan saran yang membangun guna penyempurnaan modul ini dimasa yang akan datang.

PENGESAHAN

Disusun Oleh	Diperiksa & Dikendalikan Oleh	Disetujui oleh
Irma Rofni Wulandari, S.Pd., M.Eng. Aditya Rizki Yudiantika, S.T., M.Eng. Moch Farid Fauzi, M.Kom. Deni Kurnianto Nugroho, M.Eng.	Atik Nurmasani, M.Kom	Anggit Dwi Hartanto, M.Kom
Tgl. 29 Juni 2026	Tgl.	Tgl.

Modul ini syah dan diberlakukan mulai:

Tgl.

Prof. Dr. Kusrini, M.Kom

DAFTAR ISI

(1) COVER.....	0
KATA PENGANTAR.....	1
DAFTAR ISI.....	2
1. PERTEMUAN 23 – GITHUB & COLLABORATION.....	3
1.1. Tujuan Instruksi Khusus.....	3
1.2. Alat dan Bahan.....	3
1.3. GitHub & Kolaborasi.....	3
1.3.1. Pengenalan Version Control System (VCS).....	4
1.3.2. Git vs. GitHub.....	4
1.3.3. Arsitektur Repositori: Local vs. Remote.....	4
1.3.4. Alur Kerja Kolaborasi.....	4
1.3.5. Manajemen Konflik.....	5
1.3.6. Praktik Terbaik (Best Practices).....	5
1.4. Instruksi Modul.....	5
1.4.1. Push Kode ke Remote Repository (GitHub).....	5
1.4.2. Pull Request dan Clone.....	7
1.4.3. Praktik Push dan Commit ke Akun Git.....	10
1.5. Tugas Mahasiswa.....	11
1.6. Sumber Bacaan.....	12

PERTEMUAN 23 – GITHUB & COLLABORATION

1.1. Tujuan Instruksi Khusus

Setelah mengikuti modul ini, mahasiswa diharapkan mampu:

- a. Memahami konsep dasar dan urgensi Version Control System (VCS) dalam pengembangan perangkat lunak.
- b. Menguasai alur kerja Git untuk manajemen kode sumber lokal.
- c. Mampu mengelola dan mensinkronisasikan kode antara repositori lokal dan remote repository (GitHub).
- d. Mampu mengimplementasikan alur kerja kolaborasi tim (Cloning, Push/Pull, dan Pull Request).
- e. Mampu melakukan manajemen konflik dan menjaga integritas kode sumber dalam pengembangan web.

1.2. Alat dan Bahan

- a. PC Desktop/Laptop
- b. Sistem Operasi (Windows/macOS/Linux)
- c. Git (<https://git-scm.com/download/>)
- d. VS Code atau Terminal
- e. Akun GitHub/GitLab
- f. Web Browser

1.3. GitHub & Kolaborasi

Dalam era pengembangan perangkat lunak modern, kolaborasi yang efektif dan pengelolaan kode sumber yang terstruktur menjadi pilar utama keberhasilan sebuah proyek. GitHub bukan sekadar tempat penyimpanan kode, melainkan ekosistem kolaboratif yang memungkinkan tim untuk bekerja secara sinkron, melacak perubahan dengan presisi, dan menjaga integritas proyek dari awal hingga tahap deployment. Bagian ini akan membahas bagaimana Git sebagai sistem kontrol versi dan GitHub sebagai platform kolaborasi terintegrasi untuk mendukung alur kerja pengembangan web yang profesional dan terukur.

1.3.1. Pengenalan Version Control System (VCS)

Version Control System (VCS) adalah sistem yang mencatat perubahan file dari waktu ke waktu sehingga Anda dapat memanggil versi tertentu di kemudian hari. Dalam pengembangan web, VCS sangat krusial untuk:

- Melacak riwayat perubahan (history).
- Memungkinkan kolaborasi tim tanpa menimpa pekerjaan orang lain.
- Memberikan keamanan melalui cadangan kode (backup).

1.3.2. Git vs. GitHub

Penting untuk membedakan Git dan GitHub:

- **Git**: Perangkat lunak Version Control (VCS) yang berjalan di komputer lokal (CLI/GUI).
- **GitHub**: Platform berbasis cloud yang menjadi rumah bagi repositori Git, menyediakan antarmuka untuk kolaborasi, hosting, dan manajemen proyek.

1.3.3. Arsitektur Repositori: Local vs. Remote

- **Local Repository**: Berada di komputer pengembang. Anda melakukan modifikasi di sini.
- **Remote Repository**: Berada di server pusat (seperti GitHub). Ini adalah "Source of Truth" yang menjadi titik sinkronisasi antar anggota tim.
- **Sinkronisasi**: Dilakukan melalui perintah `push` (mengirim lokal ke remote) dan `pull` (mengambil remote ke lokal).

1.3.4. Alur Kerja Kolaborasi

Dalam lingkungan tim, alur kerja standar mencakup:

- **Forking**: Membuat salinan repositori proyek orang lain ke akun pribadi Anda.
- **Cloning**: Mengunduh repositori dari remote ke lingkungan lokal.

- **Branching:** Membuat lini pengembangan terpisah (branch) agar tidak mengganggu branch utama (biasanya `main` atau `master`).
- **Committing:** Menyimpan snapshot perubahan kode di lokal.
- **Pushing:** Mengirim commit ke remote repository.
- **Pull Request (PR):** Mekanisme pengajuan perubahan dari branch fitur Anda ke branch utama untuk ditinjau oleh tim (code review) sebelum digabungkan (merge).

1.3.5. Manajemen Konflik

Konflik terjadi ketika dua orang mengubah baris kode yang sama dalam file yang sama. GitHub menyediakan fitur visual untuk mendeteksi ini. Resolusi konflik melibatkan:

- Mengidentifikasi file yang konflik.
- Membuka file tersebut, memilih perubahan mana yang dipertahankan (ours/theirs/both).
- Melakukan commit ulang untuk menandai konflik sudah selesai.

1.3.6. Praktik Terbaik (Best Practices)

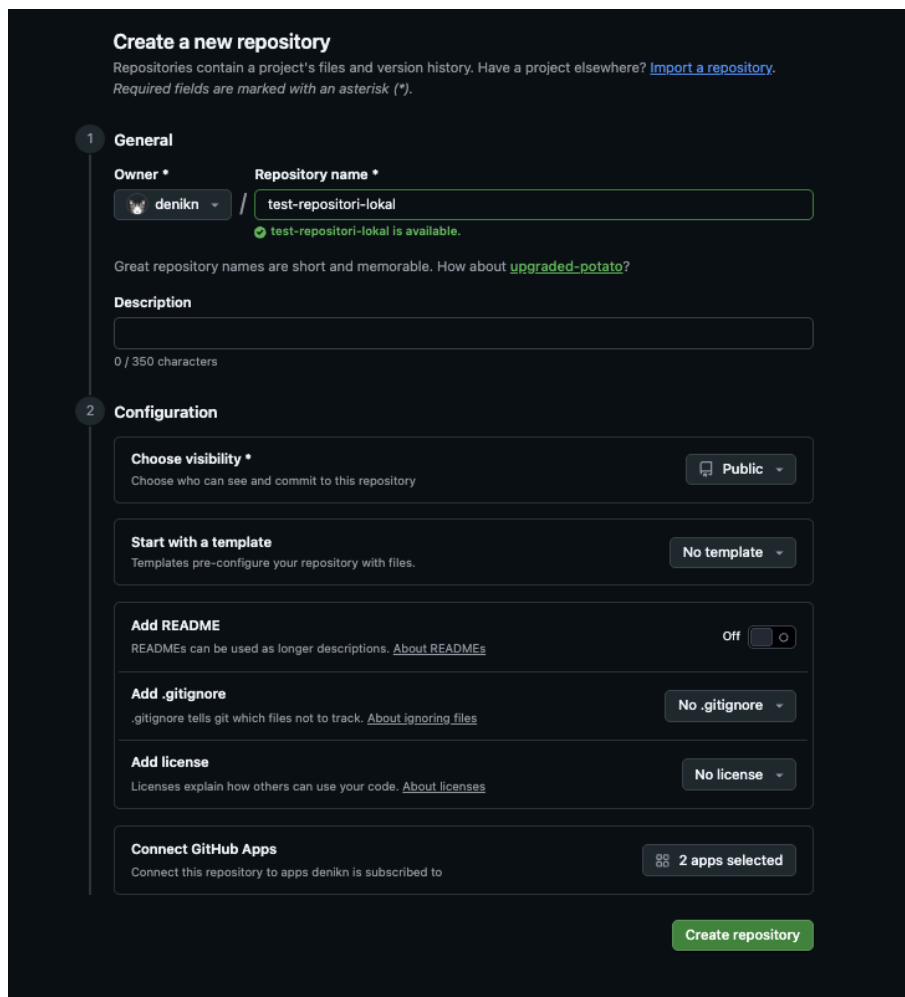
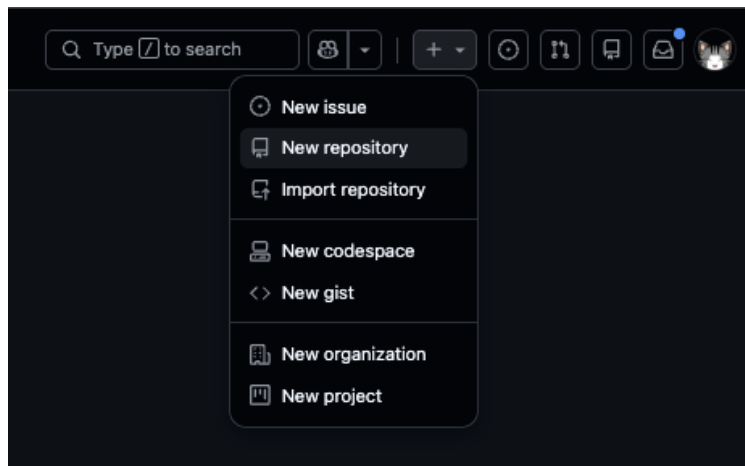
- Buat commit message yang deskriptif (gunakan imperatif, misal: "Add login feature", bukan "changed file").
- Lakukan commit sesering mungkin untuk perubahan kecil yang logis.
- Selalu `pull` kode terbaru dari remote sebelum mulai bekerja untuk menghindari konflik besar.
- Gunakan branch terpisah untuk setiap fitur baru.

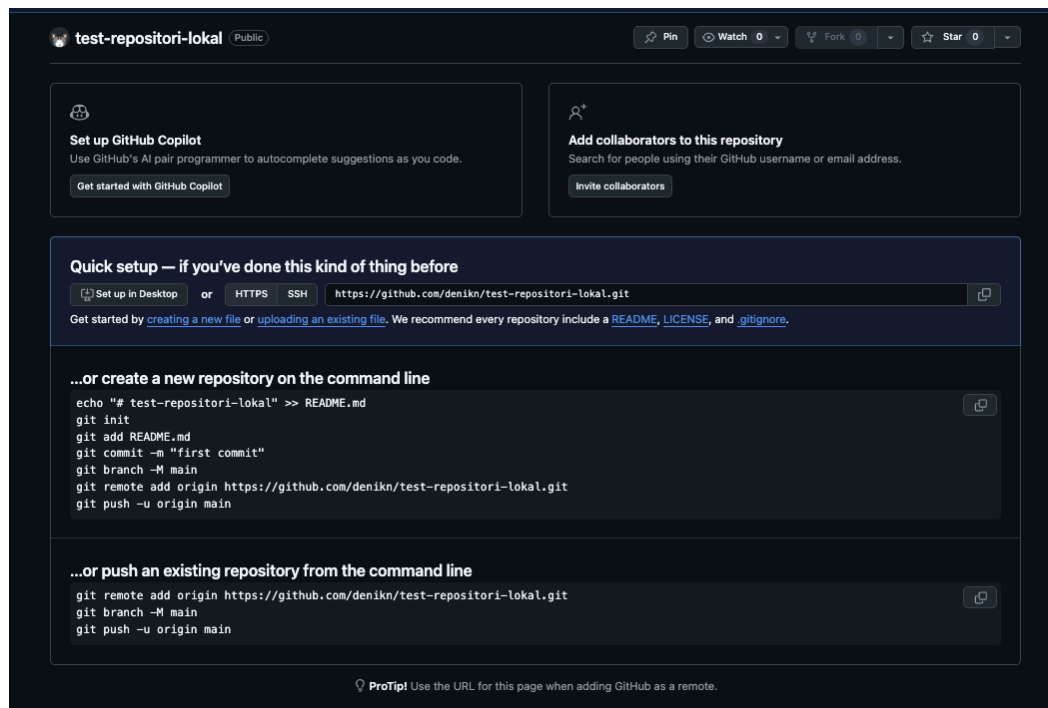
1.4. Instruksi Modul

1.4.1. Push Kode ke Remote Repository (GitHub)

Setelah Anda memiliki repositori lokal, langkah selanjutnya adalah mengirimkannya ke GitHub agar kode tersimpan di server.

1. Buat repositori baru di GitHub (pilih Public atau Private).

A screenshot of the 'Create a new repository' form in GitHub. The form is divided into two main sections: 'General' and 'Configuration'. In the 'General' section, the 'Owner' is 'denikn' and the 'Repository name' is 'test-repositori-lokal'. A green checkmark indicates that the name is available. Below this is a 'Description' field. In the 'Configuration' section, there are several options: 'Choose visibility' (set to 'Public'), 'Start with a template' (set to 'No template'), 'Add README' (set to 'Off'), 'Add .gitignore' (set to 'No .gitignore'), 'Add license' (set to 'No license'), and 'Connect GitHub Apps' (set to '2 apps selected'). A green 'Create repository' button is at the bottom right.



2. Di terminal Anda, hubungkan direktori lokal ke GitHub dengan perintah: `git remote add origin <URL_REPOSITORY_ANDA>`
3. Ubah branch default ke main (jika belum): `git branch -M main`
4. Kirim kode ke GitHub: `git push -u origin main`

```
test-local-repo git:(main) git remote add origin https://github.com/denikn/test-repositori-lokal.git
test-local-repo git:(main) x git add .
test-local-repo git:(main) x git commit -m "initial commit"
[main (root-commit) 5a47fde] initial commit
2 files changed, 1 insertion(+)
create mode 100644 .DS_Store
create mode 100644 test.txt
test-local-repo git:(main) x git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 527 bytes | 527.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/denikn/test-repositori-lokal.git
 * [new branch]      main -> main
test-local-repo git:(main) x
```

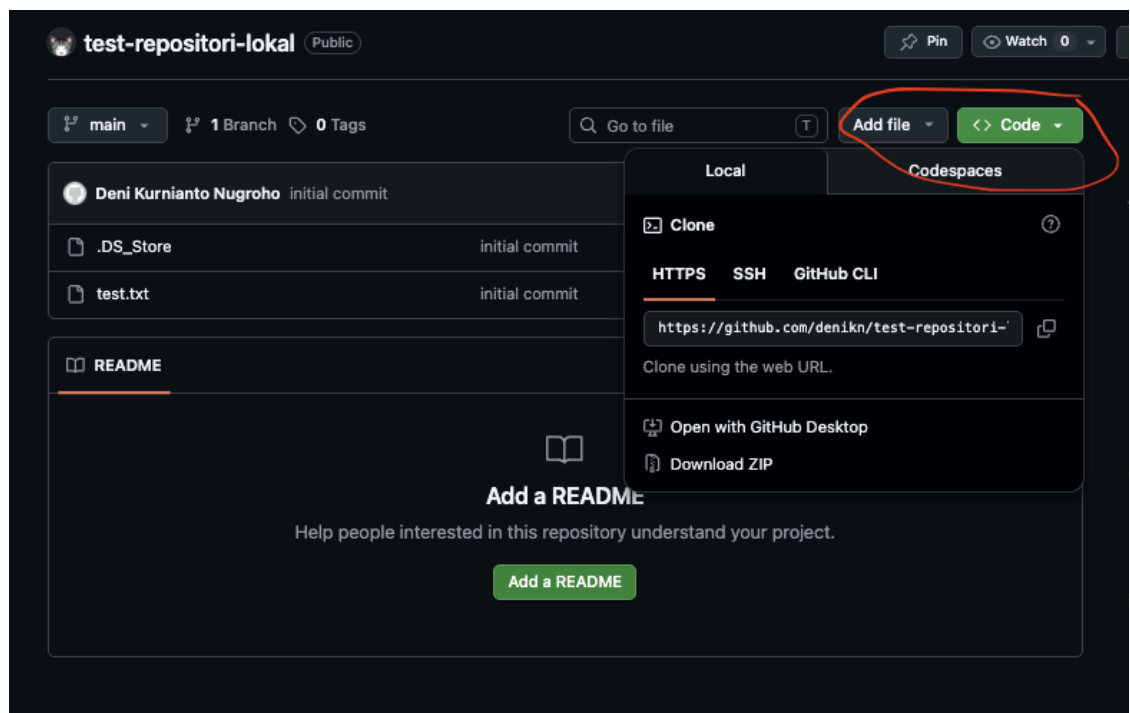
1.4.2. Pull Request dan Clone

Bagian ini menjelaskan cara mengambil kode orang lain dan mengusulkan perubahan.

A. Cloning

1. Klik tombol hijau Code di halaman repositori GitHub.

2. Salin (copy) URL yang tersedia (HTTPS).



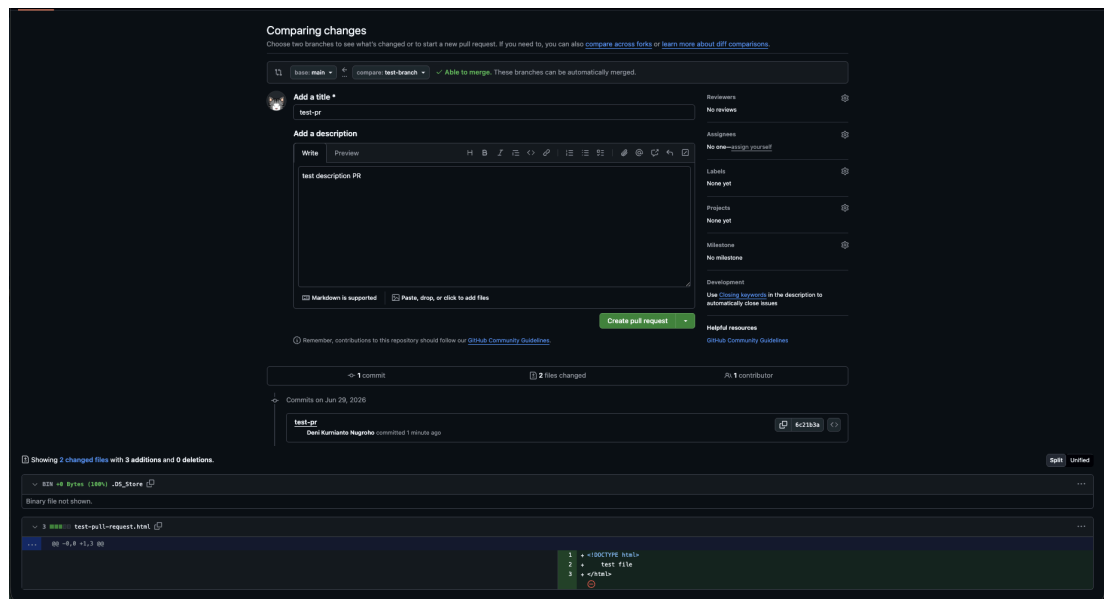
3. Di terminal, jalankan: `git clone <URL_REPOSITORY>`

B. Pull Request (PR)

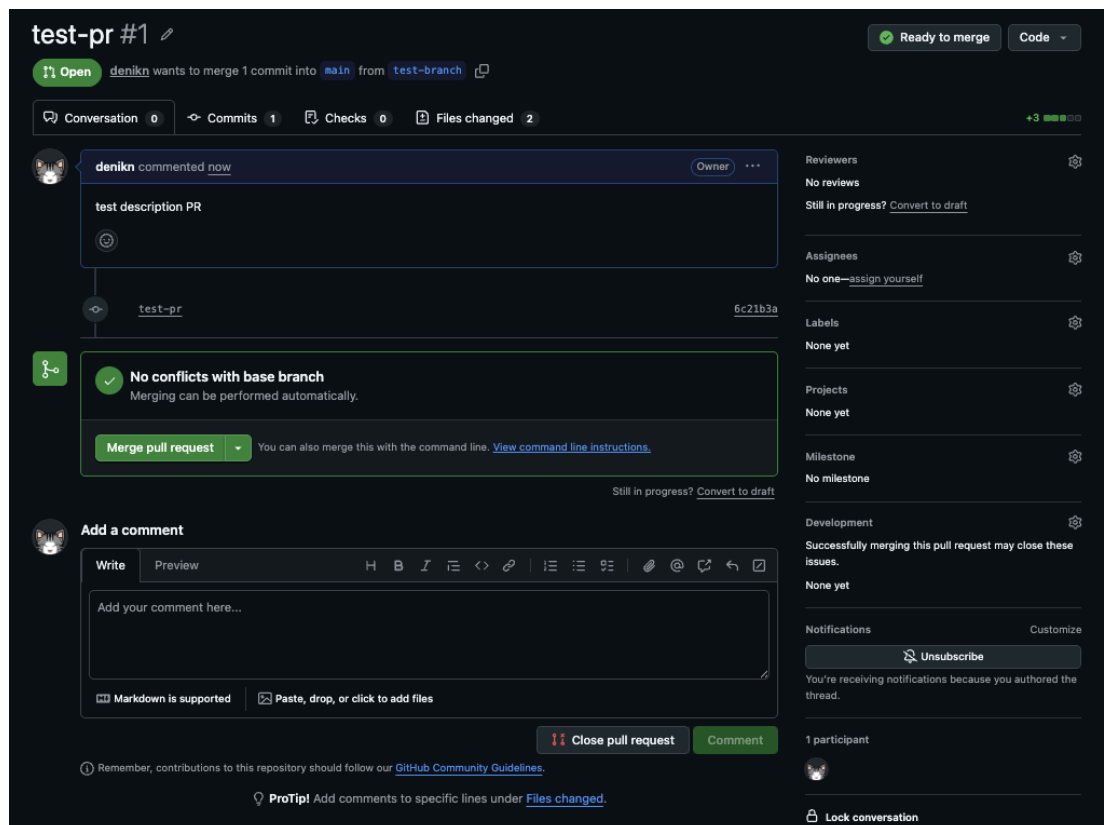
1. Lakukan perubahan pada branch baru.
2. Push perubahan tersebut ke repositori GitHub Anda.

```
test-local-repo git:(test-branch) ✖ git push origin test-branch
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 490 bytes | 490.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'test-branch' on GitHub by visiting:
remote:   https://github.com/denikn/test-repositori-lokal/pull/new/test-branch
remote:
To https://github.com/denikn/test-repositori-lokal.git
 * [new branch]      test-branch -> test-branch
test-local-repo git:(test-branch) ✖
```

3. Buka repositori di GitHub, lalu klik tombol Compare & pull request.



4. Isi deskripsi perubahan dan klik Create pull request.



1.4.3. Praktik Push dan Commit ke Akun Git

Lakukan langkah berikut sebagai latihan rutin dalam pengembangan:

1. Buat atau edit file di dalam direktori proyek Anda.
2. Periksa status file: `git status`

```
● → test-local-repo git:(test-branch) x git status
On branch test-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .DS_Store
        modified:   test-pull-request.html

no changes added to commit (use "git add" and/or "git commit -a")
○ → test-local-repo git:(test-branch) x
```

3. Siapkan file untuk dikirim: `git add .`
4. Buat commit dengan pesan yang deskriptif: `git commit -m "Deskripsi perubahan Anda"`
5. Kirim ke GitHub: `git push origin <branch anda>`

```
● → test-local-repo git:(test-branch) x git add .
● → test-local-repo git:(test-branch) x git commit -m "add changes"
[test-branch b3cc06e] add changes
 2 files changed, 1 insertion(+), 1 deletion(-)
● → test-local-repo git:(test-branch) x git push origin test-branch
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 500 bytes | 500.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/denikn/test-repositori-lokal.git
 6c21b3a..b3cc06e test-branch -> test-branch
○ → test-local-repo git:(test-branch) x
```

1.5. Tugas

Tugas ini bertujuan untuk mensimulasikan alur kerja kolaborasi tim menggunakan Git dan GitHub dalam pengembangan perangkat lunak.

Langkah-langkah Tugas:

1. Persiapan Proyek: Buat folder project baru dan masukkan script hasil latihan sebelumnya. Lakukan git init di folder tersebut.
2. Upload ke Remote: Buat repositori di GitHub, hubungkan repositori lokal Anda, lalu lakukan git push pertama kali ke branch main.
3. Kolaborasi Kelompok:
 - a. Ketua kelompok membagikan akses repositori kepada anggota tim (atau menjadikan repositori Public).
 - b. Setiap anggota kelompok melakukan git clone repositori tersebut ke laptop/perangkat masing-masing.
4. Simulasi Perubahan: Setiap anggota melakukan perubahan pada file script (misalnya menambahkan komentar atau fitur sederhana), melakukan git add, git commit, dan git push ke repositori.
5. Pelaporan: Kumpulkan laporan praktikum bersama dengan laporan di pertemuan selanjutnya. Untuk modul ini, bagian yang harus dikumpulkan adalah:
 - a. Link URL repositori GitHub.
 - b. Screenshot perintah terminal yang menunjukkan keberhasilan git push dan git log.
 - c. Nama akun GitHub tiap anggota kelompok.

1.6. Sumber Bacaan

- [1] Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. (Tersedia online di <https://git-scm.com/book/en/v2>).
- [2] GitHub. (n.d.). GitHub Docs: Understanding the GitHub flow. <https://docs.github.com/>.
- [3] Git. (n.d.). Git Documentation. <https://git-scm.com/doc>.
- [4] Atlassian. (n.d.). Git Workflow Guide. <https://www.atlassian.com/git/tutorials/comparing-workflows>.
- [5] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.